

Recap:

1 Iterating the contents of an array

- map()
- forEach()

map() -> we can transform the elements in the array as it can return a value

forEach() -> used for iteration as it does not return a value

2. push() and pop()

- push() -> to add an element in the array
- pop() -> remove the element from the end

3. filter() - filters data based on a condition

4. some() - it returns true if any of the element in the array satisfies the given condition
else returns false

5. every() - it returns true if all the elements in the array satisfies the given condition

6. reduce() - it is used to reduce the values in array to a single value

7. slice() - it is used to get the items from an array from given index till the number and
does not update the original array

8. splice() - it is similar to slice, but it can be used to add, remove elements from the original
array. meaning it can update the original array.
and returns the update / removed values.

9. shift() - it removes the first element from the array.

10. unshift() - it is used to add element at the first index

Lifecycle of a component:

3 phases in the lifecycle of a component:

1. Mounting
2. Updating
3. Unmounting

Mounting of a component:

Showing the component data on the webpage.

Updating of the component:

update the component on the webpage based on some interaction

Unmounting of the component:

removal of the component based on user interaction

To manage the lifecycle hooks and state are there.

Hooks in React

1. State
2. Lifecycle
3. Context

Hooks are special function that let us utilize the React features like state, lifecycle and context inside functional components.

Before Introduction to React Hooks

```
class Component1 extends React.Component {  
  render() {  
    return (  
      // can be used to manage lifecycle of the component  
    )  
  }  
}  
  
function Component1 = () => {  
  return(//Print Some data)  
}
```

lifecycle in class component:

Mounting = constructor -> render -> componentDidMount
Updating = shouldComponentUpdate -> render -> componentDidUpdate
Unmounting = componentWillUnmount

In order to control the lifecycle of the components hooks were introduced.
following changes were done to functional components:

1. they can access the state
2. they can use lifecycle effects - componentDidMount, componentDidUpdate
3. logic can be shared across the components
4. code becomes simple, clean and consistent

Why Hooks are useful:

1. remove the need for classes
2. 'this' keyword usage and confusion is also taken care of.
3. makes components smaller and easier to read
4. makes stateful logic reusable.
5. cleaner architecture
6. reduces complexity of lifecycle methods

what are different hooks?

1. useState

2. useEffect
3. useContext
4. useMemo
5. useCallback

What is State in React?

State is built in data structure in React that represents value that change over time and affect how the component renders and what it renders.

1. when state changes, React re-renders the component
2. it makes the UI dynamic
3. it is local and mutable using React APIs only and private to the component unless passed using the props.

Why do we need State?

UI is not static.

examples - counting of clicks

- managing toggles
- showing / hiding elements
- storing data
- form inputs

How state is created:

state is created using useState() hook.

```
const [count, setCount] = useState(0)
```

count -> name of the state

setCount -> function to update the state

useState -> it is the react hook to create the state

useState(0) -> a default value for the state

How state causes re-rendering?

1. setState function to update the state (ex. - setCount)
2. React compares the new state with the old one
3. if value is different, React re-renders only that component and its children.
 - React schedules the update and then re-renders
4. DOM is updated efficiently using the VDOM (virtual DOM)

useEffect()

lets us run side effects in the functional components

side effects = anything outside components pure rendering process

1. fetching data from API
2. DOM manipulation
3. logging

```
useEffect(() => {}, [dependencies])
```

1. Mounting phase

```
useEffect(() => {}). [])
```

when no dependency is added to the `useEffect`, it is called only once.
this is equivalent to: `componentDidMount`

2. Updating Phase

```
useEffect(() => {}, [todos])
```

it is executed when `todos` is updated
this is equivalent to = `componentDidUpdate`

3. Unmounting Phase

```
useEffect(() => {  
  return () => {}  
}, [])
```

this is equivalent to `componentWillUnmount`
why this cleanup is required?

- cancel the api calls
- cancel any subscriptions
- event listeners
- timeouts and intervals

4. For every render `useEffect` is called

```
useEffect(()=>{})
```

this is called for every update performed within the component leading to performance issues
we should avoid using this kind of `useEffect`.

In the dependency array we can pass in multiple states or state setter.

While using `useEffect` make sure that the functionalities are clubbed together based on dependencies as every `useEffect` will be called at time of rendering / updation process

DO NOT UPDATE THE SAME STATE IN THE USEEFFECT WHICH HAS THE STATE AS THE DEPENDENCY

Virtual DOM

The virtual DOM is lightweight, in-memory Javascript Object that represents the structure of a real DOM.



React does not manipulate real DOM directly during rendering. Instead it performs the following steps:

1. Builds a virtual representation (a tree of JS objects)
2. Compares with the previous virtual DOM
3. Applies the minimal change to the real DOM

